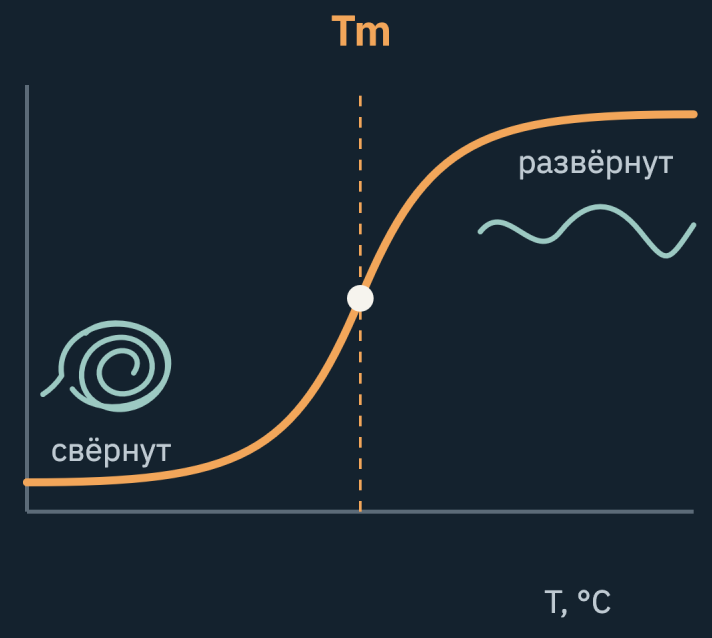
Предсказание температуры плавления белка

Введение в машинное обучение: признаки, модели, функции ошибок и метрики



ПЛАН ЗАНЯТИЯ

От последовательности до числа: шесть шагов

01

Задача

Что такое Tm и почему её предсказывают

02

ИИ, ML и DL

Типы задач и методов, примеры из биологии

03

Признаки

One-hot, ограничение по длине, аминокислотный состав

04

Модели и обучение

Линейная и логистическая регрессия, функции ошибок

05

Пайплайн и метрики

Разбиение, кросс-валидация, гиперпараметры, оценка

06

Практика

numpy, pandas, scikit-learn, XGBoost в Colab

ЗАДАЧА КУРСА

По последовательности предсказать Tm



Что такое Tm

Температура, при которой половина молекул белка развёрнута — точка перегиба кривой плавления.

Зачем предсказывать

Отбор термостабильных ферментов для биотехнологии и вариантов для дизайна — до эксперимента.

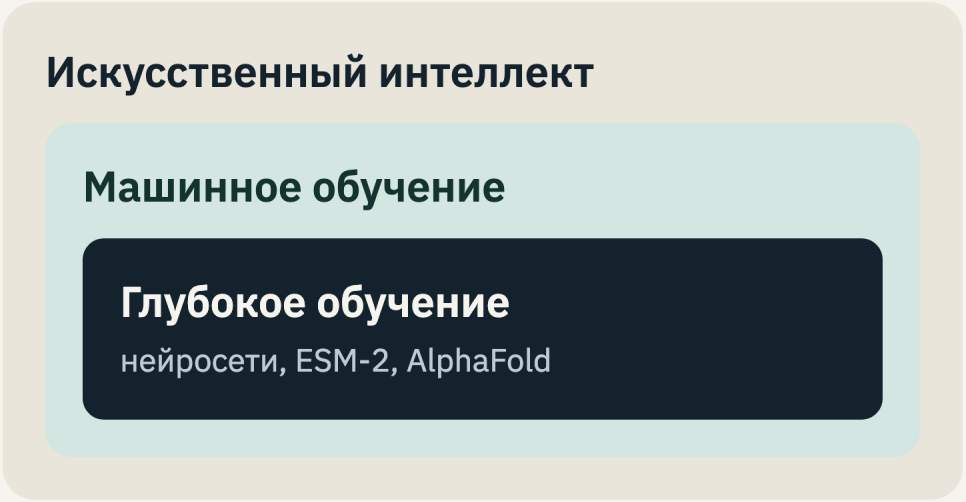
Тип задачи

Регрессия: Tm — непрерывное число. Вариант: классификация «Tm выше [60] °C или нет».

Данные протеом-масштабного термопрофилирования: Meltome Atlas, Jarzab et al., Nat Methods 2020

ПОНЯТИЯ

ИИ ⊃ машинное обучение ⊃ глубокое обучение



ИИ — любые системы для «интеллектуальных» задач, включая правила, написанные вручную.

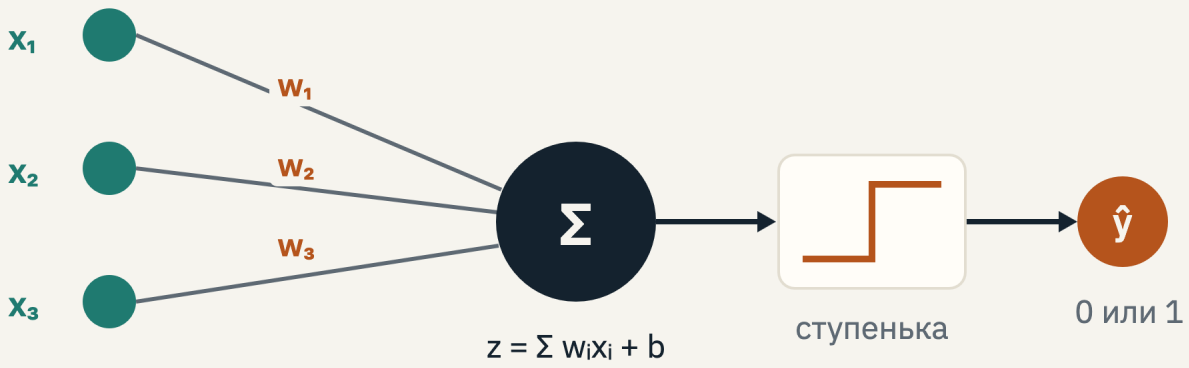
ML — модель сама подбирает параметры w по данным, чтобы f(x; w) ≈ y.

DL — ML на многослойных нейросетях, которые сами учат признаки из сырых данных.

Классический ML: признаки придумывает человек. DL: признаки выучивает сеть.

ПЕРЦЕПТРОН РОЗЕНБЛАТТА · 1958

Один нейрон: взвесить, сложить, сравнить с порогом



$\hat{y} = 1$, если $w \cdot x + b > 0$, иначе 0

Правило обучения

$w \leftarrow w + \eta \cdot (y - \hat{y}) \cdot x$

$b \leftarrow b + \eta \cdot (y - \hat{y})$

Веса меняются только на ошибках.

Теорема сходимости

Если классы линейно разделимы, за конечное число шагов найдёт разделяющую прямую.

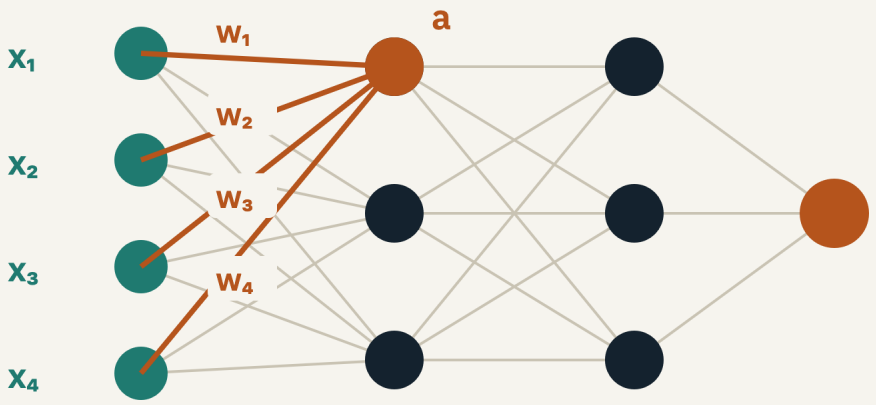
1958 · Розенблатт
перцептрон и машина Mark I, Корнелл

1969 · Минский и Пейперт
один перцептрон не решает XOR

1986 · обратное распространение
многослойные сети решают XOR

ГЛУБОКОЕ ОБУЧЕНИЕ

Нейросеть — это много слоёв простых функций



признаки x скрытые слои ŷ = Tm

Нейрон: **a = σ(w₁x₁ + w₂x₂ + w₃x₃ + w₄x₄ + b)**, где σ — нелинейность (ReLU, сигмоида).

MLP — полносвязная сеть для табличных признаков

CNN — локальные мотивы в последовательности

Трансформеры — белковые языковые модели (ESM-2), AlphaFold

Когда нужен DL: много данных и сырой вход. Иначе бустинг на хороших признаках часто не хуже.

С УЧИТЕЛЕМ И БЕЗ УЧИТЕЛЯ

Есть метки у или нет

С учителем (supervised)

Для каждого объекта известен ответ y. Модель учится на парах (x, y) и переносит это на новые белки.

Данные: последовательность + измеренная Tm.

Задачи: регрессия (Tm, °C), классификация (стабилен / нет).

Методы: линейная и логистическая регрессия, RF, XGBoost, нейросети.

Без учителя (unsupervised)

Меток y нет. Модель ищет структуру в самих данных X: группы и главные оси изменчивости.

Данные: только признаки белков, без Tm.

Задачи: кластеризация, снижение размерности.

Методы: k-means, DBSCAN, PCA, t-SNE, UMAP.

Наша задача — с учителем: метка y (Tm) есть. Без учителя пригодится, чтобы разведать и почистить данные.

КЛАССИФИКАЦИЯ ЗАДАЧ

Регрессия, классификация, кластеризация

Тип задачи	Обучение	Примеры в биологии	Методы
Регрессия: число	с учителем	Tm белка, ΔΔG мутации, аффинность лиганда (pKd), уровень экспрессии	линейная регрессия, RF, XGBoost
Классификация: класс	с учителем	патогенность мутации, локализация белка, сайт фосфорилирования, активный лиганд	логистическая регрессия, SVM, RF, XGBoost
Кластеризация: группы	без учителя	типы клеток в scRNA-seq, семейства белков, кластеры конформаций	k-means, иерархическая, DBSCAN
Снижение размерности	без учителя	визуализация экспрессии и эмбеддингов белков	PCA, t-SNE, UMAP

С учителем: есть метки y (измеренная Tm). **Без учителя:** меток нет, ищем структуру в X.

МЕТОДЫ

Основные семейства моделей

Линейные модели

Линейная и логистическая регрессия, Ridge, Lasso. Интерпретируемы, быстрые — базовая линия.

k ближайших соседей

Ответ — по самым похожим объектам. Идея как у переноса аннотации по гомологии.

SVM

Разделяющая гиперплоскость с максимальным зазором; ядра дают нелинейность.

Деревья и Random Forest

Последовательность вопросов «признак > порога?». Лес — усреднение многих деревьев.

Градиентный бустинг

XGBoost, LightGBM, CatBoost: каждое новое дерево исправляет ошибки предыдущих. Сильный выбор для табличных данных.

Нейросети

Гибкие, учат признаки сами, но требуют много данных и аккуратного обучения.

ПРИЗНАКИ

Модель видит только таблицу чисел

Белок	Длина	Гидрофоб.	E+K+R	Tm, °C
белок_1	312	0.41	0.19	52.1
белок_2	158	0.46	0.27	71.4
белок_3	540	0.37	0.16	46.8
…	…	…	…	…

↑ матрица признаков X (N × d)

↑ целевая переменная y

Значения условные, для иллюстрации

Объект — одна строка: один белок.

Признак (feature) — числовое свойство объекта: длина, доля гидрофобных остатков, заряд, pI…

Цель (target) — то, что предсказываем: Tm.

Качество признаков часто важнее выбора модели.

ONE-HOT ENCODING

Каждая аминокислота — вектор из 20 чисел с одной единицей

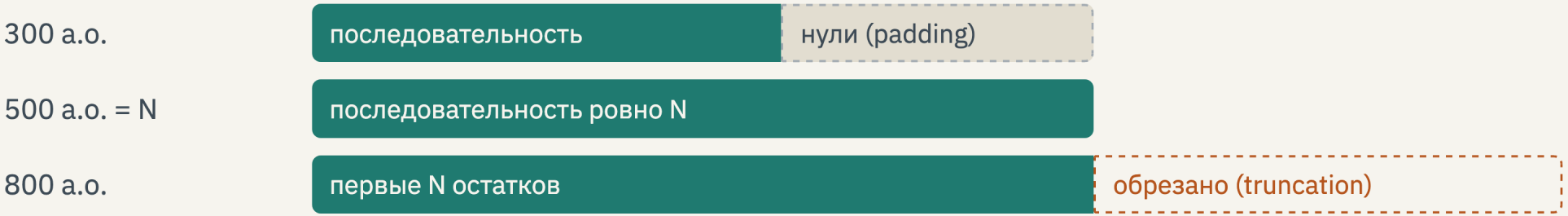
поз.	A	C	D	E	F	G	H	I	K	L	M	N	P	Q	R	S	T	V	W	Y
1 M	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
2 K	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
3 V	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
4 L	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
5 E	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Белок длины L → матрица **L × 20** → после «разворачивания» вектор из **20·L** признаков.

Почему не A=1, C=2, D=3…? Модель решит, что Y «больше» A, — ложный порядок. One-hot его убирает.

ОГРАНИЧЕНИЕ ПО ДЛИНЕ

Модели нужен вектор фиксированной длины



Обрезка и дополнение до N — просто, но теряем хвосты и плодим «пустые» признаки.

Скользящее окно — предсказываем по фрагментам и усредняем.

Признаки, не зависящие от длины — аминокислотный состав, физико-химия. Сегодняшний выбор.

Эмбеддинги языковых моделей + усреднение по позициям. У ESM-2 предел ≈ 1022 остатка.

PADDING

Padding — дополнение коротких последовательностей нулями



■ реальные остатки · □ padding — добавленные нули до общей длины L = 6

Зачем: модели нужен вход одной длины, а белки разной. Нули достраивают короткие до общей L.

Маска: отдельный вектор помечает, где настоящие остатки, а где нули — чтобы модель их не путала.

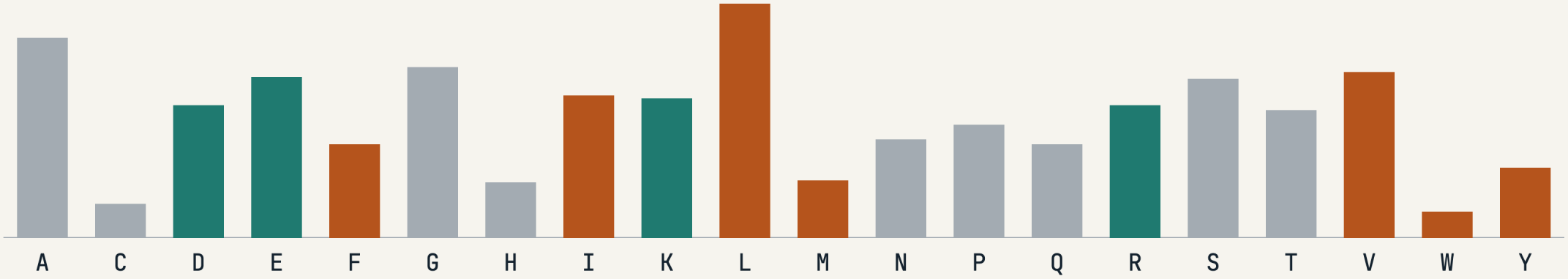
Для аминокислотного состава padding не нужен: там всегда 20 признаков при любой длине.

ONE-HOT БЕЗ ПОЗИЦИИ

Сумма one-hot по позициям = аминокислотный состав

$$f_a = (1/L) \cdot \Sigma_i x_{ia}$$

Складываем строки матрицы L × 20 и делим на L: **всегда 20 признаков** при любой длине. Порядок остатков теряется.



■ гидрофобные (F, I, L, M, V, W, Y) · ■ заряженные (D, E, K, R) · пример: средний состав белков, ≈ по UniProt

ПОЧЕМУ СОСТАВ РАБОТАЕТ

Термостабильность оставляет след в составе

Гидрофобные: I, V, L, F, M, W, Y

Формируют ядро. Больше гидрофобных — плотнее упаковка и выше стабильность.

Заряженные: E, K, R

Солевые мостики и ионные сети на поверхности; у белков термофилов их доля выше.

Термолабильные: N, Q, C, M

N и Q дезамидируются, C и M окисляются при нагреве — у термофилов их меньше.

Сигнатура IVYWREL

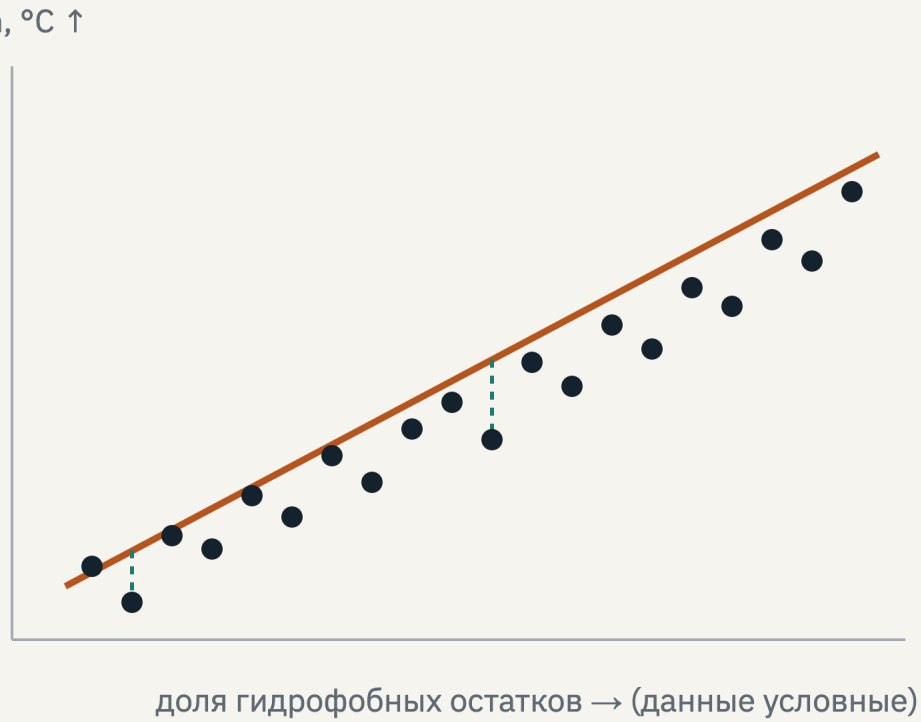
Доля этих 7 остатков в протеоме сильно коррелирует с оптимальной температурой роста организма.

Ограничение. Состав не видит порядок и структуру. Часть сигнала — просто «из какого организма белок»: Tm сильно зависит от температуры роста вида.

IVYWREL: Zeldovich, Berezovsky, Shakhnovich, PLoS Comput Biol 2007

МОДЕЛЬ 1 · РЕГРЕССИЯ

Линейная регрессия



$$\hat{y} = w_0 + w_1 x_1 + \ldots + w_d \cdot x_d$$

Обучение: подобрать веса w, минимизируя $MSE = (1/N)\cdot\Sigma(y_i - \hat{y}_i)^2$; пунктир — ошибки отдельных точек.

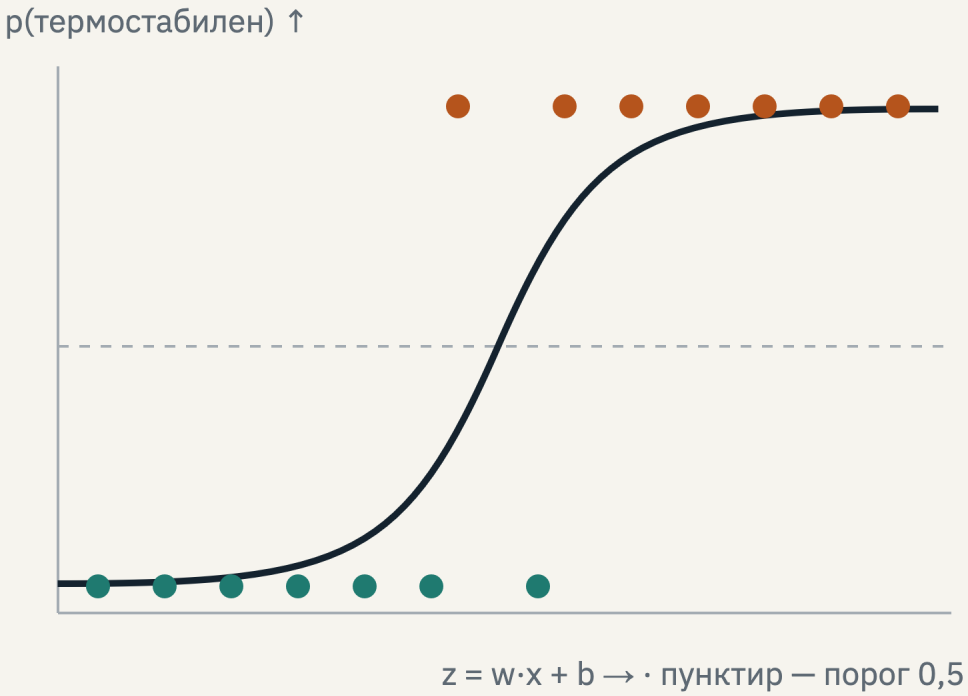
Интерпретация: знак w_j — направление влияния признака, величина — сила (если признаки масштабированы).

Регуляризация: Ridge (L2) и Lasso (L1) штрафуют большие веса и снижают переобучение.

`LinearRegression(), Ridge(alpha=1.0)`

МОДЕЛЬ 2 · КЛАССИФИКАЦИЯ

Логистическая регрессия



z = w·x + b → · пунктир — порог 0,5

$$p = \sigma(z) = 1 / (1 + e^{(-z)})$$

Та же линейная комбинация признаков, но «сжатая» сигмоидой в вероятность от 0 до 1.

Пример: класс 1 — Tm ≥ [60] °C, класс 0 — ниже. Предсказываем класс 1, если p > 0,5.

Функция ошибок: log-loss (бинарная кросс-энтропия). Несмотря на название — это **классификация**.

```
LogisticRegression(C=1.0)
```

ИЗ РЕГРЕССИИ В КЛАССИФИКАЦИЮ

Из числа Tm — в три класса

психрофильные	мезофильные	термофильные
Tm < 45 °C	45–65 °C	Tm > 65 °C

белок_2: Tm = 71,4 °C → **термофильный**

Пороги здесь — пример. Их задаём сами: по квантилям Tm (чтобы классы были сбалансированы) или по температуре роста организма-источника.

Цель
число → класс из трёх

Модель
LogisticRegression, XGBClassifier

Функция ошибок
MSE → кросс-энтропия

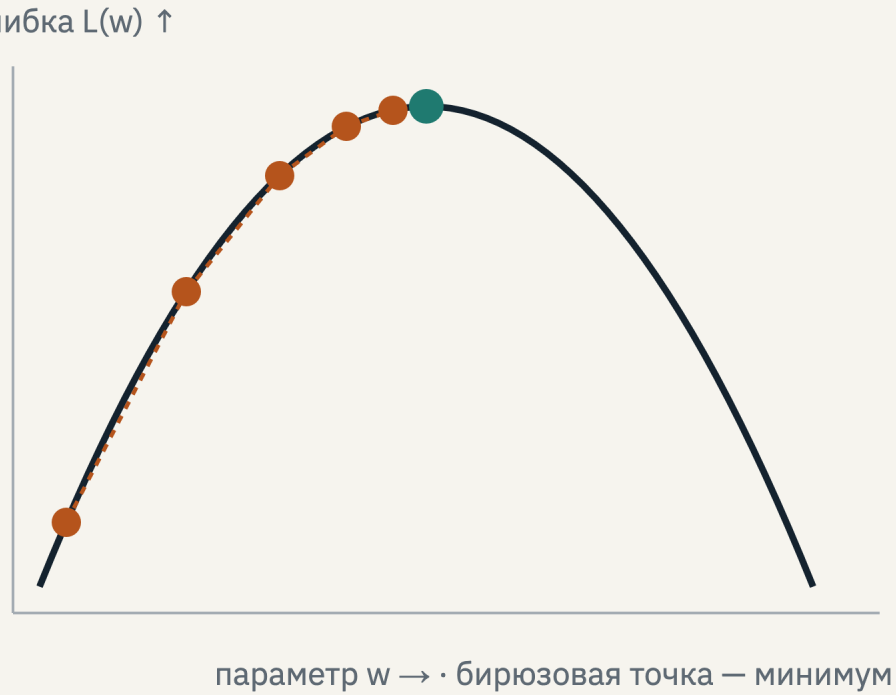
Метрики
F1, MCC, матрица ошибок 3 × 3

```
df["cls"] = pd.cut(df["tm"], bins=[-np.inf, 45, 65, np.inf], labels=["психро", "мезо", "термо"])
```

Цена: теряем информацию — 64,9 и 65,1 °C попадают в разные классы, а 46 и 64 °C — в один.

ОБУЧЕНИЕ

Обучение — это минимизация функции ошибок



$$L(w) = (1/N)\cdot\Sigma\ \ell(y_i, f(x_i; w)) \to \min$$

$$w \leftarrow w - \eta\cdot\nabla L(w)$$

Градиентный спуск: считаем, куда ошибка растёт быстрее всего, и делаем шаг в обратную сторону.

η — скорость обучения (learning rate): мала — обучение медленное, велика — шаги «перепрыгивают» минимум.

Все модели курса учатся так или похоже: от линейной регрессии до нейросетей; бустинг шагает по градиенту в пространстве деревьев.

ФУНКЦИИ ОШИБОК

Функция ошибок задаёт, что считать «плохим» ответом

Функция	Формула	Задача	Особенности
MSE	$(1/N)\cdot\Sigma(y - \hat{y})^2$	регрессия	сильно штрафует большие ошибки, чувствительна к выбросам
MAE	$(1/N)\cdot\Sigma\lvert y - \hat{y}\rvert$	регрессия	устойчива к выбросам, в единицах y (°C)
Huber	MSE для малых ошибок, MAE для больших	регрессия	компромисс между ними
Log-loss	$-[y\cdot\log p + (1 - y)\cdot\log(1 - p)]$	бинарная классификация	логистическая регрессия, нейросети
Кросс-энтропия	$-\Sigma_k\, y_k\cdot\log p_k$	много классов	с softmax на выходе
Hinge	$\max(0, 1 - y\cdot z)$	классификация	SVM

Регуляризация добавляется к любой из них: $+ \lambda\cdot\|w\|^2$ (L2) или $+ \lambda\cdot\|w\|_1$ (L1).

ПАЙПЛАЙН

Обычный пайплайн машинного обучения

1

Предобработка

Очистка, дубликаты, признаки, масштабирование

2

Разбиение

train / test, например 80/20; тест откладываем

3

Выбор модели

Сначала простая базовая: среднее, линейная регрессия

4

Обучение

С параметрами по умолчанию: model.fit(X_train, y_train)

5

Кросс-валидация

k-fold на train: средняя метрика и её разброс

6

Гиперпараметры

Grid / Random search, Optuna — внутри CV

7

Итоговая модель

Обучаем на всём train с лучшими параметрами

8

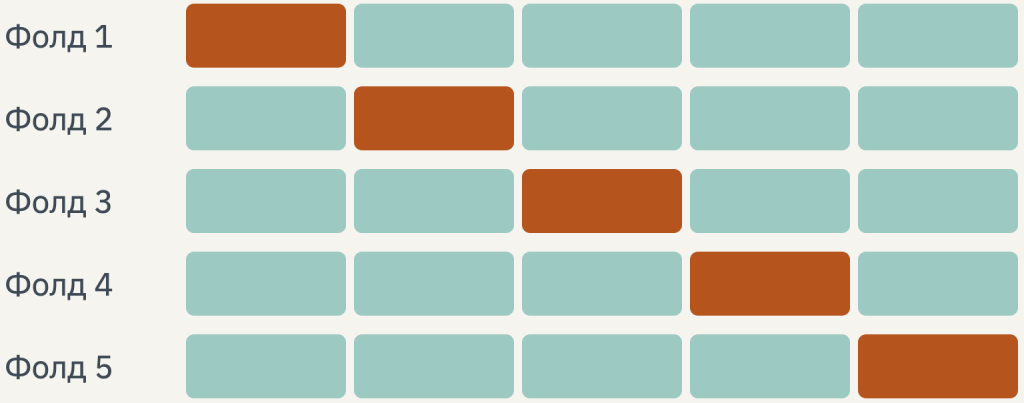
Оценка на тесте

Один раз, в самом конце — честная метрика

Шаги 3–6 повторяются, пока модель улучшается. Тест не участвует ни в одном решении.

ВАЛИДАЦИЯ

Кросс-валидация и утечка данных



■ обучение · ■ проверка · итог: среднее ± разброс по 5 фолдам

Все пять частей по очереди становятся проверочными — оценка не зависит от одного удачного разбиения.

Утечка через гомологию

Близкие гомологи в train и test: модель «узнаёт» белок, а не предсказывает. Метрики завышены.

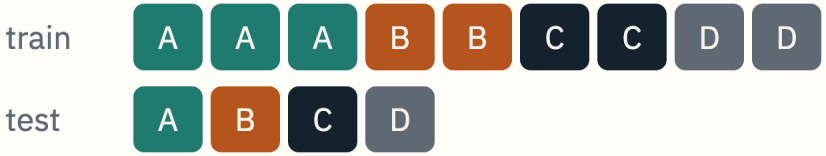
Решение: кластеризовать по идентичности последовательностей (MMseqs2, CD-HIT, порог [30–40] %) и делить по кластерам — GroupKFold.

Ещё одна утечка: масштабирование и отбор признаков подгоняем только на train — удобно через sklearn Pipeline.

УДАЛЕНИЕ ГОМОЛОГОВ · ЗАЧЕМ

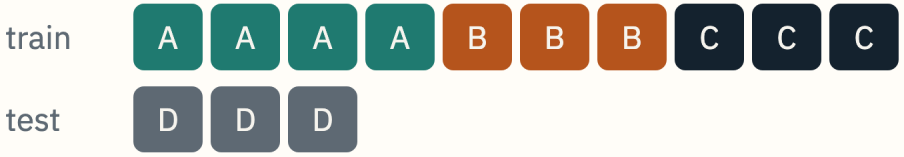
Гомологи в train и test завышают метрики

Случайное разбиение



Каждое семейство есть и в train, и в test → модель «узнаёт» белки

Разбиение по кластерам



В test только новое семейство → честная оценка

A–D — семейства гомологичных белков

Гомологи похожи по последовательности и структуре, а часто и по свойствам. В больших наборах много ортологов одного белка из разных видов.

Утечка: если близкий гомолог тестового белка есть в train, модель отвечает «как у соседа» — работает как поиск по сходству, а не выучивает закономерности.

Перекос: большие семейства с десятками копий перевешивают при обучении.

Нас интересует качество на **новых** белках — значит, test должен быть непохож на train.

УДАЛЕНИЕ ГОМОЛОГОВ · КАК

Кластеризуем по идентичности: MMseqs2 и CD-HIT

MMseqs2

Быстрый, справляется с миллионами последовательностей и с низкой идентичностью (30 % и ниже).

```
mmseqs easy-cluster in.fasta clu tmp \
  --min-seq-id 0.3 -c 0.8
```

--min-seq-id — порог идентичности, -c — покрытие. Выход: clu_cluster.tsv (представитель → член кластера).

CD-HIT

Классический жадный алгоритм; для белков порог идентичности не ниже 40 %.

```
cd-hit -i in.fasta -o nr40.fasta \
  -c 0.4 -n 2
```

-c — порог, -n — длина слова: 2 при c = 0.4–0.5, 5 при c ≥ 0.7. Выход: nr40.fasta и nr40.fasta.clstr.

Вариант 1 · убрать избыточность

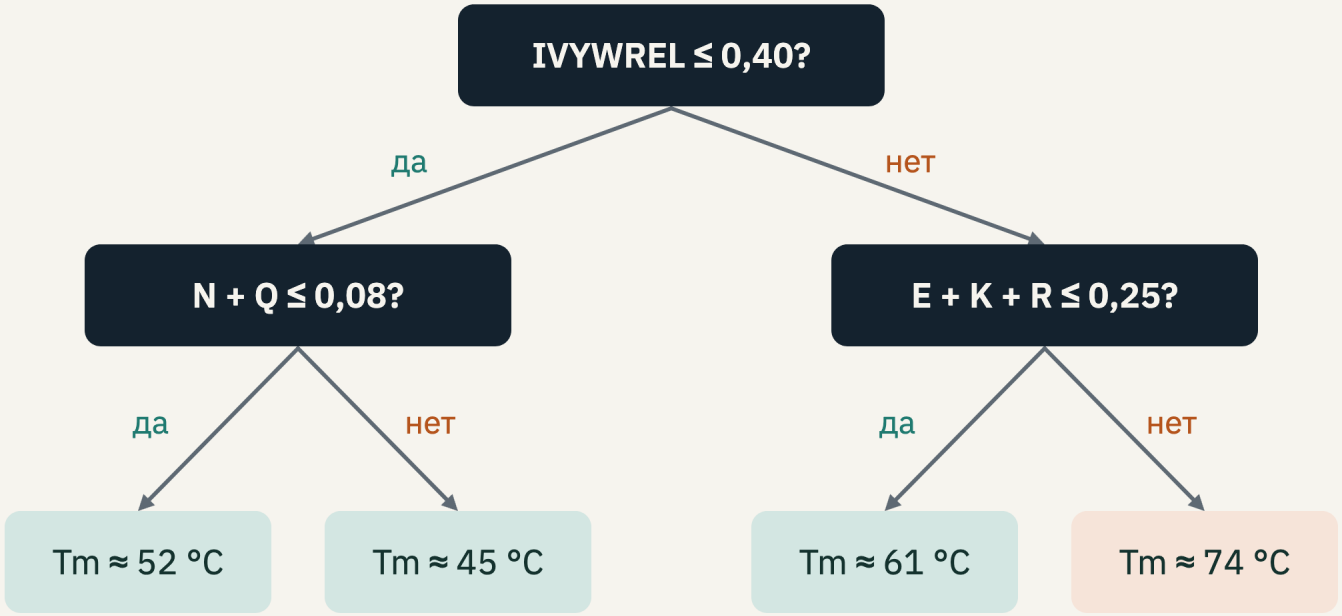
Оставить одного представителя на кластер или усреднить Tm по кластеру.

Вариант 2 · делить по кластерам

`GroupKFold(5).split(X, y, groups=clusters)`

РЕШАЮЩЕЕ ДЕРЕВО

Дерево — цепочка вопросов о признаках



Узел — условие на признак, **лист** — ответ: среднее Tm обучающих белков, попавших в лист. При обучении дерево в каждом узле перебирает признаки и пороги и берёт разбиение, сильнее всего уменьшающее MSE. Значения условные.

Гиперпараметры дерева

Параметр	Что задаёт
max_depth	число уровней (здесь 2)
min_samples_split	минимум белков, чтобы делить узел
min_samples_leaf	минимум белков в листе
max_features	сколько признаков пробовать в узле
ccp_alpha	обрезка лишних ветвей

Глубокое дерево с листьями по 1 белку = переобучение.

Random Forest — много деревьев на случайных подвыборках, ответ усредняется. **XGBoost** — деревья строятся по очереди и исправляют ошибки предыдущих.

ГИПЕРПАРАМЕТРЫ

Параметры учатся, гиперпараметры выбираем мы

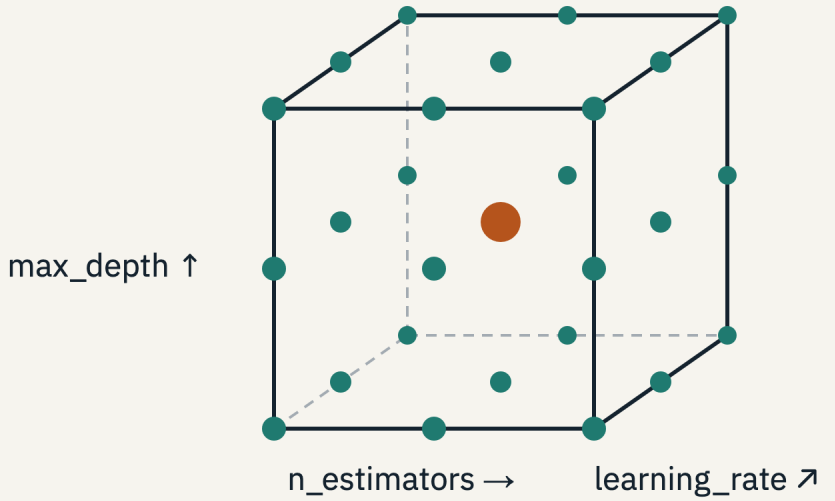
XGBoost	Что делает	Обычно
n_estimators	число деревьев	100–2000
max_depth	глубина дерева	3–10
learning_rate	вклад каждого дерева	0.01–0.3
subsample	доля объектов на дерево	0.5–1.0
colsample_bytree	доля признаков на дерево	0.5–1.0
reg_lambda	L2-регуляризация	0–10

Grid search — все узлы сетки (точки на кубе)

Random search — случайные точки внутри куба

Optuna — следующую точку выбирает по прошлым результатам

Переобучение: ошибка на train падает, на валидации растёт. Меньше глубину, больше регуляризацию.



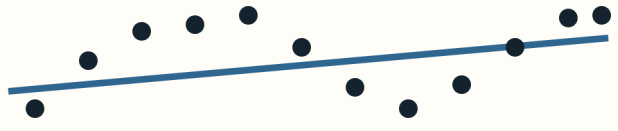
● 27 точек сетки 3 × 3 × 3 · ● лучшая по CV

Каждая точка — одна комбинация и одно обучение с CV. Шесть гиперпараметров по 5 значений — уже 15 625 точек.

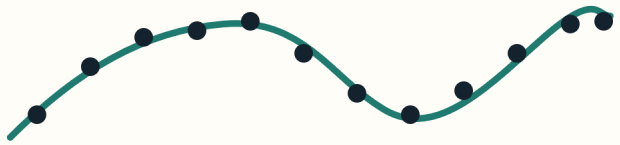
НЕДООБУЧЕНИЕ И ПЕРЕОБУЧЕНИЕ

Модель должна поймать закономерность, а не шум

Недообучение · слишком простая

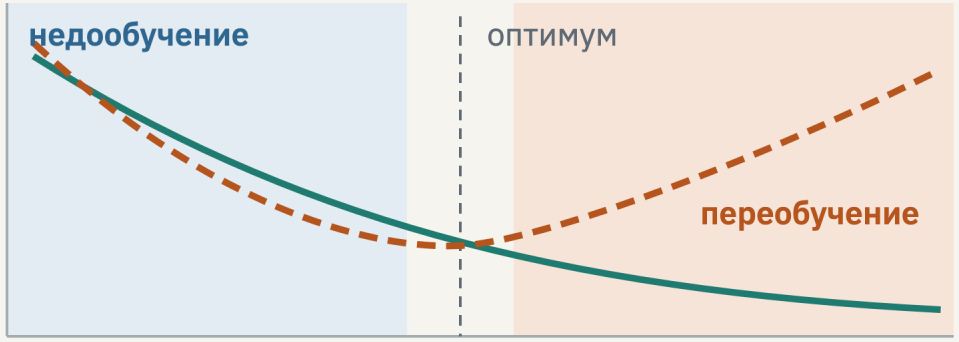


Хорошо · ловит закономерность



Переобучение · запоминает шум





ошибка ↑ · сложность модели → (глубина, эпохи, число признаков)

— train · - - валидация

Недообучение: обе ошибки высокие

Сложнее модель, больше признаков, меньше регуляризации, дольше учить.

Переобучение: train ≪ валидация

Больше данных, регуляризация (L1/L2, dropout), меньше max_depth, early stopping.

ФУНКЦИЯ ОШИБОК ≠ МЕТРИКА

Учим по одному, оцениваем по другому

Функция ошибок

Её минимизирует алгоритм при обучении. Должна быть гладкой и дифференцируемой — нужен градиент.

MSE, кросс-энтропия, Huber

Метрика

По ней мы оцениваем модель на валидации и тесте. Должна быть понятна человеку; градиент не нужен.

MAE в °C, F1, MCC, Spearman ρ

Задача	Учим по	Отчитываемся
Регрессия Tm	MSE	MAE, R^2, Spearman ρ
Классы Tm	кросс-энтропия	F1, MCC, ROC-AUC



МЕТРИКИ · РЕГРЕССИЯ

Насколько хорошо мы предсказываем Tm

Метрика	Формула	Смысл
MAE	$(1/N) \cdot \Sigma \lvert y_i - \hat{y}_i \rvert$	средняя ошибка в °C: «ошибаемся на столько-то градусов»
RMSE	$\sqrt{[\ (1/N) \cdot \Sigma\ (y_i - \hat{y}_i)^2\ ]}$	тоже в °C, сильнее штрафует большие промахи; RMSE ≥ MAE
R^2	$1 - \Sigma\ (y_i - \hat{y}_i)^2 / \Sigma\ (y_i - \bar{y})^2$	доля объяснённой дисперсии: 1 — идеал, 0 — как среднее, бывает < 0
Pearson r	$\mathrm{cov}(y, \hat{y}) / (\sigma_y \cdot \sigma_{\hat{y}})$	линейная связь; не видит систематический сдвиг
Spearman ρ	r Пирсона на рангах; без совпадений: $1 - 6 \cdot \Sigma\ d_i^2 / (N \cdot (N^2 - 1))$, d_i — разность рангов	совпадение порядка белков — важно для отбора самых стабильных

Базовая линия: DummyRegressor — всегда среднее Tm. Модель должна быть заметно лучше.

МЕТРИКИ · КЛАССИФИКАЦИЯ

Матрица ошибок и метрики на её основе

	предсказано 1	предсказано 0
факт 1	**TP**	**FN** ошибка II рода
факт 0	**FP** ошибка I рода	**TN**

Бирюзовые — верные ответы, бежевые — ошибки

Ловушка дисбаланса: если 95 % белков — класс 0, модель «всегда 0» даёт accuracy 0,95 и бесполезна.

Метрика	Формула / смысл
Accuracy	(TP + TN) / все — доля верных
Precision	TP / (TP + FP) — сколько найденных верны
Recall	TP / (TP + FN) — сколько истинных нашли
F1	2·P·R / (P + R) — баланс precision и recall
ROC-AUC	площадь под кривой TPR(FPR) по всем порогам = $P(score^{+} > score^{-})$; 0,5 — случайно
MCC	$(TP \cdot TN - FP \cdot FN) / \sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}$ — устойчива к дисбалансу; от −1 до 1

ROC-AUC И ОШИБКИ I И II РОДА

Порог решает, какую ошибку мы допускаем

TPR = Recall = 1 − β ↑



FPR = α = 1 − specificity →

■ AUC ≈ 0,84 · пунктир — случайно, 0,5 · ● текущий порог

I род · FP · α

«Ложная тревога»: назвали белок термостабильным, а он нет — зря потратим эксперимент.

FPR = FP / (FP + TN)

II род · FN · β

«Пропуск»: не заметили настоящий термостабильный белок — потеряли кандидата.

TPR = TP / (TP + FN)

ROC-кривая: двигаем порог вероятности от 1 до 0 и для каждого ставим точку (FPR, TPR). Ниже порог — меньше пропусков, но больше ложных тревог.

AUC — площадь под кривой = вероятность, что случайный положительный белок получит score выше случайного отрицательного. 0,5 — монетка, 1 — идеал. Не зависит от выбора порога.

```
roc_curve(y, p) · roc_auc_score(y, p)
```

Ещё метрики: дисбаланс, вероятности, много классов

Метрика	Формула	Когда полезна
Specificity	$TN / (TN + FP)$	доля верно найденных отрицательных
Balanced accuracy	(Recall + Specificity) / 2	accuracy, честная при дисбалансе
F-beta	$(1 + \beta^2) \cdot P \cdot R / (\beta^2 \cdot P + R)$	$\beta > 1$ — важнее recall, $\beta < 1$ — precision
PR-AUC (AP)	$\Sigma_n (R_n - R_{n-1}) \cdot P_n$ — площадь под кривой precision–recall	редкий положительный класс
Log-loss	$-(1/N) \cdot \Sigma [y_i \cdot \log p_i + (1 - y_i) \cdot \log(1 - p_i)]$	качество вероятностей, а не только классов
Cohen’s κ	$(p_o - p_e) / (1 - p_e)$; p_o — доля совпадений, p_e — случайных	согласие сверх случайного
Macro / micro F1	macro — среднее F1 по классам; micro — F1 по сумме TP, FP, FN всех классов	3+ класса (психро / мезо / термо)

ИНСТРУМЕНТЫ

Четыре библиотеки на сегодня

numpy

Массивы и линейная алгебра. Матрица признаков X — это numpy-массив.

```
np.array
np.eye(20)
X.mean(axis=0)
```

pandas

Таблицы (DataFrame): загрузка, очистка, группировка данных о Tm.

```
pd.read_csv
df.dropna()
df.groupby()
```

scikit-learn

Модели, разбиение, кросс-валидация, метрики — единый интерфейс fit / predict.

```
train_test_split
cross_val_score
GridSearchCV
```

xgboost

Градиентный бустинг деревьев. Совместим с API scikit-learn.

```
XGBRegressor
XGBClassifier
.feature_importances_
```

Всё уже установлено в Google Colab: import numpy as np, pandas as pd

ЖЕЛЕЗО

CPU и GPU: мощные ядра или тысячи простых

CPU · 4–64 мощных ядра

управление, предсказание ветвлений, большой кэш

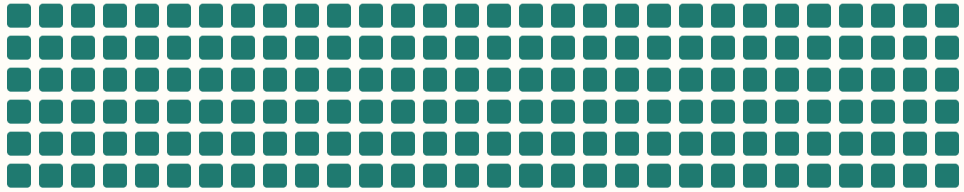


Сложная логика и последовательные алгоритмы: pandas, парсинг FASTA, Monte Carlo в Rosetta.

16 одинаковых операций на 4 ядрах → 4 шага

GPU · тысячи простых ядер

общее управление, мало кэша



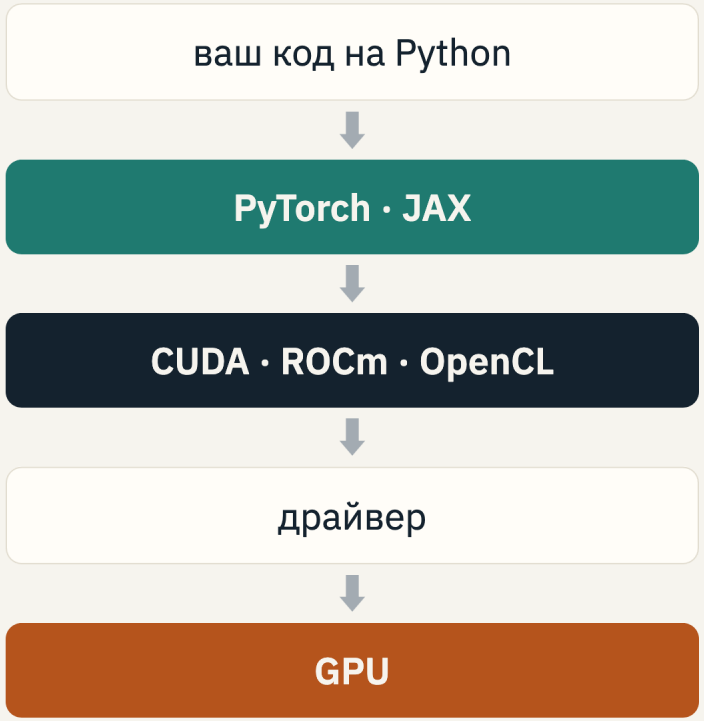
Одна операция над огромным массивом чисел: умножение матриц в нейросетях.

16 одинаковых операций → 1 шаг, все сразу

Нейросеть ≈ умножение матриц → на GPU в десятки–сотни раз быстрее. **Но** данные надо скопировать в память GPU (tensor.to("cuda")): для маленьких задач копирование съедает выигрыш.

ПРОГРАММНЫЙ СТЕК

PyTorch и JAX пишем мы, CUDA — под капотом



Что	Зачем	Кто
PyTorch	Фреймворк для нейросетей: тензоры, автоградиент, слои. Код выполняется сразу — легко отлаживать	Meta, PyTorch Foundation
JAX	Функциональный фреймворк: grad, jit (компиляция через XLA), vmap. CPU, GPU, TPU	Google
CUDA	Платформа программирования GPU NVIDIA + библиотеки cuBLAS, cuDNN. Только NVIDIA	NVIDIA
OpenCL	Открытый стандарт того же уровня: GPU AMD, Intel, NVIDIA, CPU, FPGA. В DL почти не используется	Khronos Group

PyTorch: ESM-2, Boltz, OpenFold. **JAX:** AlphaFold2, ColabFold, ColabDesign (BindCraft).

```
torch.cuda.is_available() · model.to("cuda")
```

ПРАКТИКА В COLAB

Минимальный пайплайн: две модели и CV

```
import numpy as np, pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression
from xgboost import XGBRegressor

AA = "ACDEFGHIKLMNPQRSTVWY"
def composition(seq): # 20 долей аминокислот
    return np.array([seq.count(a) / len(seq) for a in AA])

df = pd.read_csv("tm_data.csv") # колонки: sequence, tm
X = np.vstack(df["sequence"].map(composition)); y = df["tm"].values
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)

for model in [LinearRegression(), XGBRegressor(n_estimators=500, max_depth=4, learning_rate=0.05)]:
    mae = -cross_val_score(model, X_tr, y_tr, cv=5, scoring="neg_mean_absolute_error")
    print(type(model).__name__, f"CV MAE = {mae.mean():.1f} ± {mae.std():.1f} °C")
```

tm_data.csv — [ссылка на данные курса] · X_te, y_te не трогаем до финальной оценки

ДАЛЬШЕ

Как улучшить результат

Богаче признаки

Дипептидный состав (400), шкалы гидрофобности, заряд, pI, длина.

Языковые модели

Эмбеддинги ESM-2, усреднённые по позициям, + регрессия поверх.

Структура

Модель AlphaFold → доступность, контакты, солевые мостики, оценки стабильности (Rosetta, FoldX).

Учесть организм

Температура роста вида как признак или раздельная оценка по видам.

Данные и валидация

Чистка выбросов, усреднение повторов, разбиение по кластерам гомологов.

Модель

Подбор гиперпараметров, ансамбли моделей, затем нейросети.

Правило: меняем одну вещь за раз и сравниваем на одной и той же кросс-валидации.

ПЛАН НА СЕМЕСТР

Одна задача — Tm — через всё усложняющиеся модели

1	**Основы Python и ML**	признаки, модели, метрики — сегодня
2	**One-hot encoding**	последовательность как числа, состав и позиции
3	**Первая нейронная сеть**	MLP, обучение, padding и маски
4	**Предобученные трансформеры**	эмбеддинги ESM-2 + регрессия поверх
5	**Трансформеры с доп. параметрами**	SaProt (структурный алфавит), METL
6	**Своя языковая модель**	RNN / LSTM с нуля

ИТОГИ И ЗАДАНИЕ

Модель видит числа — значит, всё решают признаки и честная проверка

Признаки

Аминокислотный состав снимает проблему длины и уже несёт сигнал стабильности.

Обучение

Минимизация функции ошибок; сначала базовая модель, потом сложнее.

Оценка

Кросс-валидация без утечки, тест — один раз, MAE и Spearman.

Задание в Colab: посчитать состав, обучить LinearRegression и XGBRegressor, сравнить MAE на 5-fold CV с DummyRegressor.